Manual Optimization: Timed Results of Index vs Pointer

Emmanuel Cano

Objective

- Analyze compiler-generated assembly for two functions:
    - ClearUsingIndex()
    - ClearUsingPointers()
- Compare performance on ARM (Raspberry Pi) and x86-64 (Linux Intel) by using the chrono library to time results of different sized data.
- Compare original vs. optimized versions for both functions
- Apply manual optimizations and explain why pointer-based loops are more easily optimized by compilers and hardware.

C++ Code For Both Systems: ClearUsingIndex()

```
#include <iostream>
#include <chrono>
using namespace std;
using namespace std::chrono;

void ClearUsingIndex(int Array[], int size) {
    for (int i = 0; i < size; i++)
        Array[i] = 0;
}

int main() {
    // Array sizes to test
    int sizes[] = {10000, 100000, 1000000, 10000000, 100000000};

    for (int s = 0; s < 5; s++) {
        int size = sizes[s];

        // Dynamically allocate array of given size
        int* Array = new int[size];

        // Fill the array with dummy values
        for (int i = 0; i < size; i++)
            Array[i] = i;

        // Start timing
        auto start = high_resolution_clock::now();

        // Clear the array using index
        ClearUsingIndex(Array, size);

        // End timing
        auto end = high_resolution_clock::now();

        // Compute duration
        auto duration = duration_cast<nanoseconds>(end - start);
```

```
        cout << size << "," << duration.count() << endl;

        delete[] Array;
    }

    return 0;
}
```

C++ Code For Both Systems: ClearUsingPointers()

```
#include <iostream>
#include <chrono>
using namespace std;
using namespace std::chrono;

void ClearUsingPointers(int* Array, int size) {
    int* p;
    for (p = &Array[0]; p < &Array[size]; p++)
        *p = 0;
}

int main() {
    // Array sizes to test
    int sizes[] = {10000, 100000, 1000000, 10000000, 100000000};

    for (int s = 0; s < 5; s++) {
        int size = sizes[s];

        // Dynamically allocate array of given size
        int* Array = new int[size];

        // Fill the array with dummy values
        for (int i = 0; i < size; i++)
            Array[i] = i;

        // Start timing
        auto start = high_resolution_clock::now();

        // Clear the array using pointers
        ClearUsingPointers(Array, size);

        // End timing
        auto end = high_resolution_clock::now();

        // Compute duration
        auto duration = duration_cast<nanoseconds>(end - start);

        cout << size << "," << duration.count() << endl;

        delete[] Array;
    }

    return 0;
}
```

Methodology

Experimental Setup

All experiments were conducted on two platforms: a Raspberry Pi 5 (ARMv8-A) and a Linux Intel system (x86-64). Both systems executed the same C++ implementations of an array clearing approach, index and pointer, to ensure functional equivalence across architectures. Timing measurements focused exclusively on the execution of the function clearing for each array size: 10000, 100000, 1000000, 10000000, 100000000.

Table 1. System Configuration

Component	Linux Intel (Specification)	Raspberry Pi (Specification)
Operating System	Ubuntu 25.10 (64-bit)	Debian GNU/Linux 12 (bookworm)
Processor	13th Gen Intel(R) Core(™) i7-1355U	Cortex-A76
Architecture	x86_64	aarch64
Compiler	g++ 15.2.0	g++ 12.2.0
C++ Standard	C++ 17	C++ 17

Compilation Strategy

All C++ programs were compiled using the GNU C++ compiler (g++) with default compilation settings, with no explicit optimization flags specified.
By omitting explicit optimization flags (e.g., -O1, -O2, -O3), the compiler produced unoptimized baseline code equivalent to -O0. This approach was intentionally chosen to:

- Expose compiler-generated stack-based loop structures
- Preserve explicit load, store, and branch instructions
- Facilitate direct inspection of the generated machine code
- Prevent automatic compiler optimizations from obscuring performance characteristics

Assembly output for each program was generated using:

g++ -S

The resulting .s files served as the baseline for manual inspection and optimization.

Manual Assembly Optimization

Manual optimizations were applied directly to the compiler-generated assembly (.s) files. These optimizations focused on:

- Reducing unnecessary memory accesses
- Eliminating redundant instructions
- Improving register utilization
- Simplifying control flow where possible

Optimized assembly files were reassembled and linked to produce new executables. These optimized binaries were then evaluated under the same execution conditions as the original versions to ensure fair comparison.

Timing Methodology

Execution time was measured within each C++ program using the C++ standard library:

- std::chrono::high_resolution_clock was used to record start and end timestamps
- Elapsed time was computed using std::chrono::duration_cast to nanoseconds
- Only the code region under evaluation was timed

Timing results were printed to standard output for collection by an external script.

Automation and Repetition

A Python script was used to automate execution and data collection:

- Each executable was run 10 times
- The script recorded individual execution times
- The average execution time was calculated automatically for each test

This approach reduced manual error, improved repeatability, and helped mitigate noise from transient system effects.

Data Visualization

All timing results were visualized using Matplotlib:

- Charts compare original and manually optimized implementations
- Execution time is reported in nanoseconds
- Average values are shown, with raw timing data retained for reference

Data visualization was performed entirely through Python scripts to ensure consistency and reproducibility.
Methodological Limitations

Measurements reflect the execution time of a single invocation of the evaluated code per run. Although repeated execution and averaging reduce variability, results may still be influenced by:

- Timing overhead from std::chrono
- Cache state and branch prediction
- Operating system scheduling effects

Despite these limitations, the methodology is sufficient for relative performance analysis, which is the primary objective of this project.

Index vs Pointer Timed Results

Table 2. Raspberry Pi - Index vs. Pointer Performance (Average)

Array Size	Index (ns)	Pointer (ns)
10000	25980.1	31367
100000	409188	381882.5
1000000	5294660.9	4169062.3
10000000	50468566.6	42594075
100000000	508715519.1	422950686.1

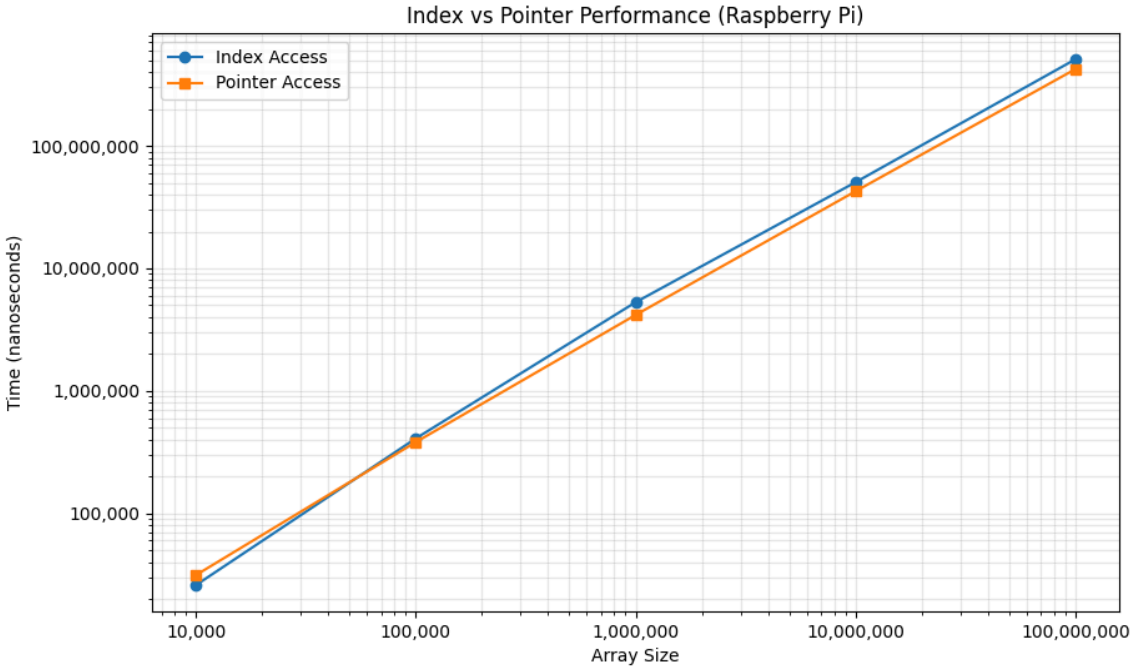

Figure 1. *Raspberry Pi timing comparison of index-based vs pointer-based array clearing functions using compiler-generated code with no optimization flags.*

Table 3. Linux Intel - Index vs. Pointer Performance (Average)

Array Size	Index (ns)	Pointer (ns)
10000	15792	9126.4
100000	75273.3	98875.6
1000000	1075911.8	1477055.7
10000000	15422548	15488939
100000000	153523296.1	122375688.2

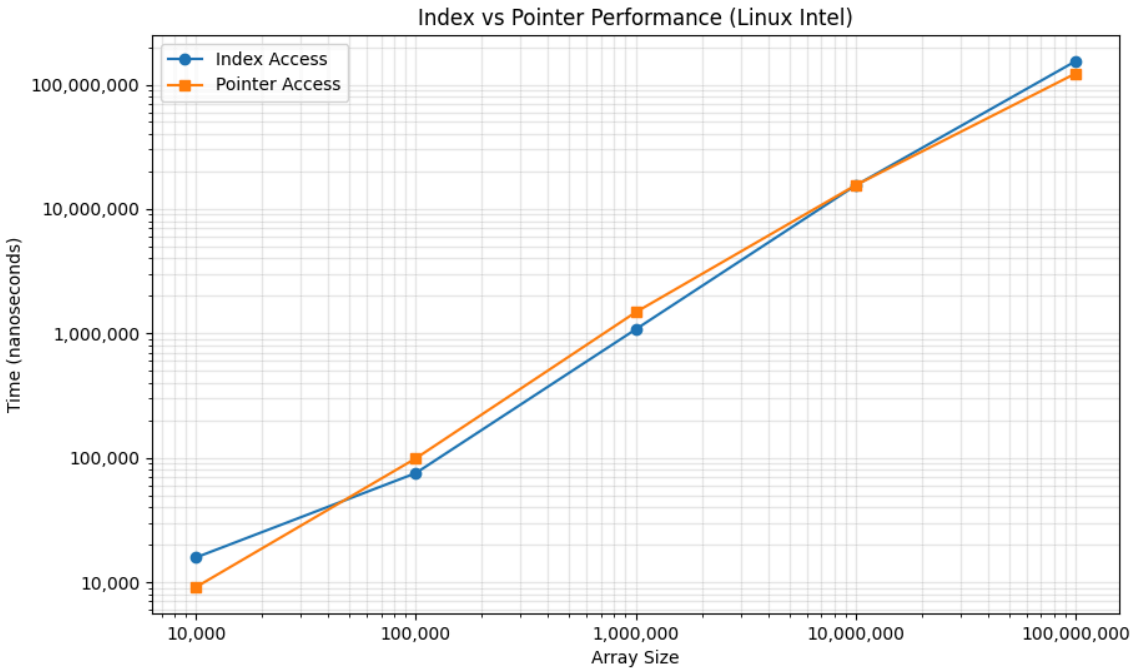

Figure 2. *Linux Intel timing comparison of index-based vs pointer-based array clearing functions using compiler-generated code with no optimization flags.*

Why Pointers Are Faster (General Concept)

Table 4. Index and Pointer Concepts

Mechanism	Index Loop	Pointer Loop
Address Calculation	Base + (i x element size) each iteration	Auto-incremented pointer
Register Use	Needs extra register for i and multiply	Single Pointer Register
CPU Micro-ops	Load + Multiply + Add	Single Add

In general pointer arithmetic is faster than index arithmetic, yet the test results for Linux Intel shows that index arithmetic is faster. After analyzing the original s file machine code for Linux Intel, it can be seen that the index code was faster because:

- The loop counter (i) was kept in a register.

The pointer code was slower because:

- It kept the loop's current pointer in stack
- It recalculated the loop limit address on every single iteration.

The compiler generated a sub-optimal s file but we can manually optimize it.

Raspberry Pi Performance Analysis

Raspberry Pi s File (Snippet): Original ClearUsingIndex()

```
	.text
	.align 2
	.global	_Z15ClearUsingIndexPii
	.type	_Z15ClearUsingIndexPii, %function
_Z15ClearUsingIndexPii:
.LFB2046:
	.cfi_startproc
	sub	sp, sp, #32
	.cfi_def_cfa_offset 32
	str	x0, [sp, 8]
	str	w1, [sp, 4]
	str	wzr, [sp, 28]
	b	.L5
.L6:
	ldrsw	x0, [sp, 28]
	lsl	x0, x0, 2
	ldr	x1, [sp, 8]
	add	x0, x1, x0
	str	wzr, [x0]
	ldr	w0, [sp, 28]
	add	w0, w0, 1
	str	w0, [sp, 28]
.L5:
	ldr	w1, [sp, 28]
	ldr	w0, [sp, 4]
	cmp	w1, w0
	blt	.L6
	nop
```

```
      nop
      add    sp, sp, 32
      .cfi_def_cfa_offset 0
      ret
      .cfi_endproc
.LFE2046:
      .size  _Z15ClearUsingIndexPii, .-_Z15ClearUsingIndexPii
```

Code Snippet Explanation

- `ldrsw` : loads signed 32-bit i and extends to 64-bit.
- `lsl x0, x0, 2` : multiplies index by 4 (each int = 4 bytes).
- `ldr x1, [sp, 8]` : retrieves base address of array.
- `add x0, x1, x0` : computes element address.
- `str wzr, [x0]` : stores 0 in that array element (wzr = zero register).
- `ldr w0, [sp, 28]` : Loads i from stack.
- `add w0, w0, 1` : It increments i.
- `str w0, [sp, 28]` : Stores it back to the stack
- `ldr w1, [sp, 28]` : Loads i from stack.
- `ldr w0, [sp, 4]` : Loads size from stack.
- `cmp w1, w0` : Compare i to size
- `blt .L6` : if i < size → branch to .L6 to write the next element.

Issues:

- It repeatedly loads and stores variable i from memory.
- It uses 64-bit arithmetic when 32-bit would suffice.
- It reloads Array from memory on every loop iteration.

Raspberry Pi s File (Snippet): Optimized ClearUsingIndex()

```
_Z15ClearUsingIndexPii:
    // Optimized ClearUsingIndex for ARMv8 (Raspberry Pi)
    // int Array[], int size

    cbz     w1, .Ldone
    mov     x2, x0
    mov     w3, w1

.Lloop:
    str     wzr, [x2], #4
    subs    w3, w3, #1
    bne     .Lloop

.Ldone:
    ret

    .size   _Z15ClearUsingIndexPii, .-_Z15ClearUsingIndexPii
```

```

```

Explanation of Optimized Code Block

- `cbz w1, .Ldone` : Skip everything if array length = 0
- `mov x2, x0` : Copy base address into a new pointer register
- `mov w3, w1` : Keeps `size` in a counter register
- `str wzr, [x2], #4` : Store 0, then increment address by 4 bytes (post-increment)
- `subs w3, w3, #1` : Decrement counter and set condition flags
- `bne .Lloop` : Loop while not equal to 0
- `ret` : Return when finished

Results:

- This version eliminates stack operation and all unnecessary loads/stores operations.

Raspberry Pi s File (Snippet): Original ClearUsingPointers()

```
	.text
	.align 2
	.global	_Z18ClearUsingPointersPii
	.type	_Z18ClearUsingPointersPii, %function
_Z18ClearUsingPointersPii:
.LFB2046:
	.cfi_startproc
	sub	sp, sp, #32
	.cfi_def_cfa_offset 32
	str	x0, [sp, 8]
	str	w1, [sp, 4]
	ldr	x0, [sp, 8]
	str	x0, [sp, 24]
	b	.L5
.L6:
	ldr	x0, [sp, 24]
	str	wzr, [x0]
	ldr	x0, [sp, 24]
	add	x0, x0, 4
	str	x0, [sp, 24]
.L5:
	ldrsw	x0, [sp, 4]
	lsl	x0, x0, 2
	ldr	x1, [sp, 8]
	add	x0, x1, x0
	ldr	x1, [sp, 24]
	cmp	x1, x0
	bcc	.L6
	nop
	nop
	add	sp, sp, 32
	.cfi_def_cfa_offset 0
	ret
```

```
      .cfi_endproc
.LFE2046:
      .size  _Z18ClearUsingPointersPii, .-_Z18ClearUsingPointersPii
```

Code Snippet Explanation

- `sub sp, sp, #32` : Makes stack space for locals
- `str x0, [sp, 8]` : Save the first argument (`Array`)
- `str w1, [sp, 4]` : Save the second argument (`size`)
- `ldr x0, [sp, 8] / str x0, [sp, 24]` : p = Array
- `.L5` : start of loop condition
- `ldrsw x0, [sp, 4] / lsl x0, x0, 2` : Compute `size * sizeof(int)
- `ldr x1, [sp, 8] / add x0, x1, x0` : Compute `&Array[size]`
- `ldr x1, [sp, 24] / cmp x1, x0` : Compare `p < &Array[size]
- `bcc .L6` : If true, continue loop body
- `.L6` : Loop body
- `ldr x0, [sp, 24] / str wzr, [x0]` : *p = 0
- `add x0, x0, 4 / str x0, [sp, 24]` : p++

Issues:

- It uses stack memory for everything ([sp, ...]) instead of keeping values in registers.
- It recomputes &Array[size] every loop iteration which is unnecessary.
- It uses 64-bit register arithmetic (x registers) even though pointers to int are fine as 64-bit values, but reloading them from memory every iteration adds cost.
- No loop unrolling or memset optimization.

Raspberry Pi s File (Snippet): Optimized ClearUsingPointers()

```
_Z18ClearUsingPointersPii:
.LFB2046:
      .cfi_startproc
      cbz    w1, .Ldone

      mov    x2, x0
      mov    x3, x1

.Lloop:
      stur   wzr, [x2]
      add    x2, x2, #4
      subs   x3, x3, #1
      b.ne   .Lloop
.Ldone:
      ret
      .cfi_endproc
.LFE2046:
      .size  _Z18ClearUsingPointersPii, .-_Z18ClearUsingPointersPii
```

Explanation of Optimized Code Block

- `cbz w1, .Ldone` : If size == 0, return immediately
- `mov x2, x0` : x2 = current pointer
- `mov x3, x1` : x3 = remaining count
- `stur wzr, [x2]` : Store 0 at *ptr
- `add x2, x2, #4` : Advance pointer by 4 bytes (int)
- `subs x3, x3, #1` : Decrement counter
- `b.ne .Lloop` : Continue until zero

Results:

- Removed unnecessary stack operations keeping all values in registers (x0, x1, x2). The old version repeatedly stored and loaded from [sp, ...], wasting cycles.
- Constant computation outside loop: &Array[size] computed once.
- Uses post-increment store: str wzr, [x0], #4 increments pointer automatically. Used subs + b.ne for compact decrementing loop.
- Fewer instructions per iteration: 3 vs ~10 before.
- Early exit if size == 0 via cbz.

Raspberry Pi - Index vs Pointer Timed Results (Optimized)

Table 5. Raspberry Pi (Optimized) - Index vs. Pointer Performance (Average)

Array Size	Index (ns)	Pointer (ns)
10000	4466.5	4392.4
100000	43852	43794.4
1000000	439751.9	436189
10000000	4348767.2	4391183.8
100000000	42904818.7	42597956.2

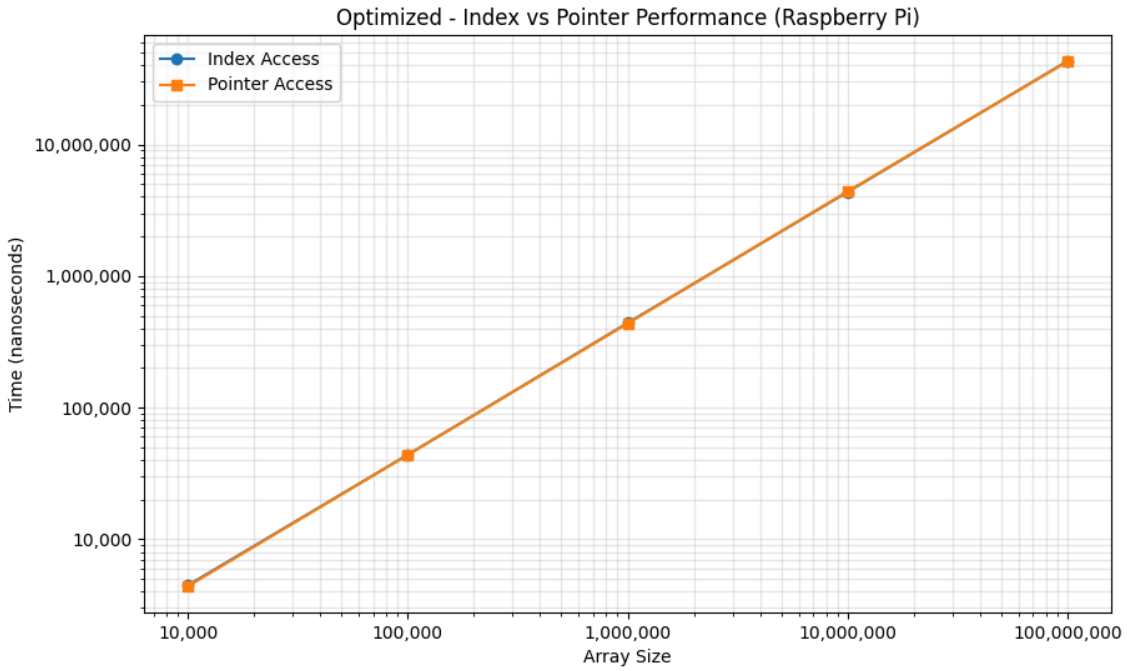

Figure 3. *Raspberry Pi timing comparison of index-based vs pointer-based array clearing functions using manual optimization of compiler-generated code with no optimization flags.*

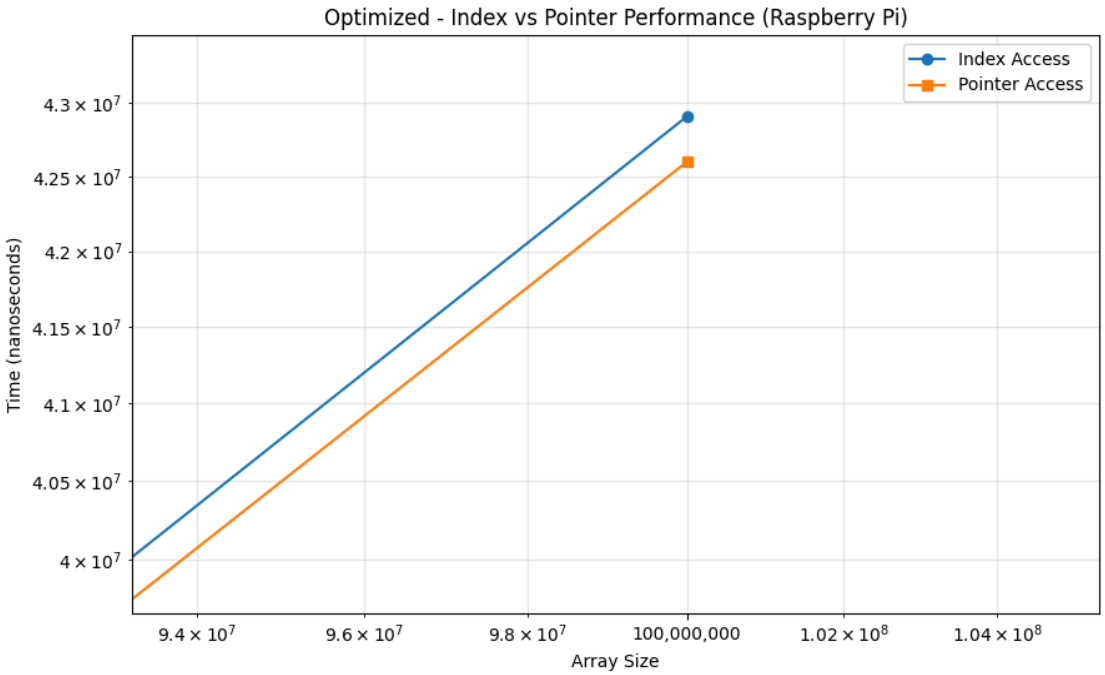

Figure 4. *Zoom-in of figure 3 for better visualization of Raspberry Pi timing comparison of index-based vs pointer-based array clearing functions using manual optimization of compiler-generated code with no optimization flags.*

Table 6. Raspberry Pi - Index vs. Index (Optimized) Performance (Average)

Array Size	Original (ns)	Optimized (ns)
10000	25980.1	4466.5
100000	409188	43852
1000000	5294660.9	439751.9
10000000	50468566.6	4348767.2
100000000	508715519.1	42904818.7

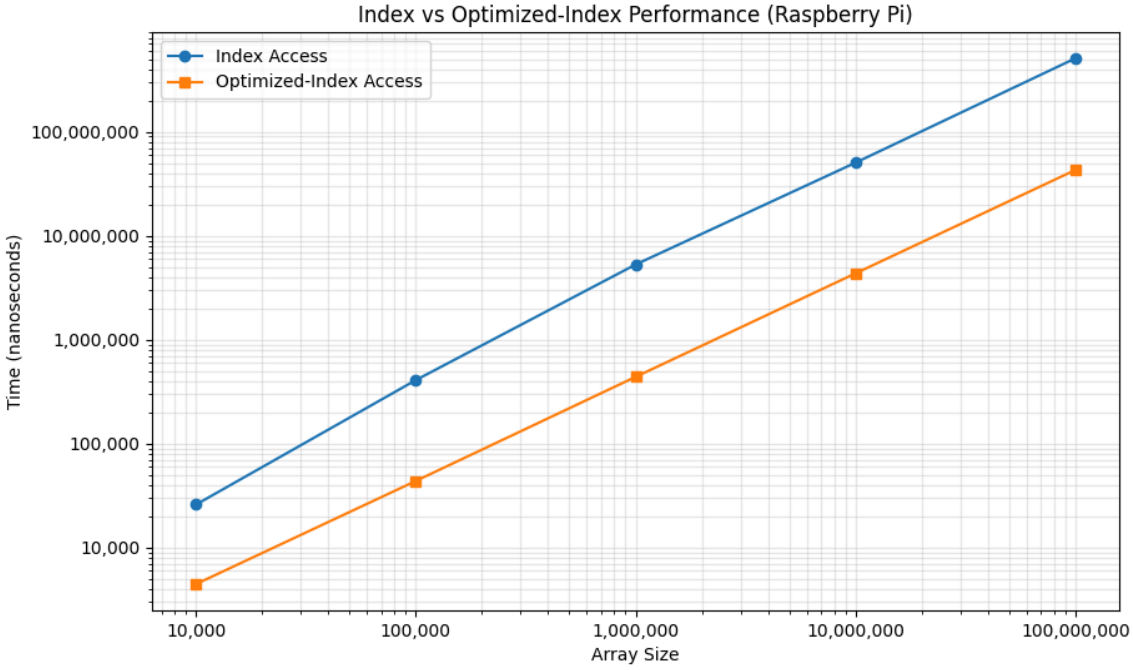

Figure 5. *Raspberry Pi timing comparison of index-based vs manually optimized index-based array clearing functions of compiler-generated code with no optimization flags.*

Table 7. Raspberry Pi - Pointer vs. Pointer (Optimized) Performance (Average)

Array Size	Original (ns)	Optimized (ns)
10000	31367	4392.4
100000	381882.5	43794.4
1000000	4169062.3	436189
10000000	42594075	4391183.8
100000000	422950686.1	42597956.2

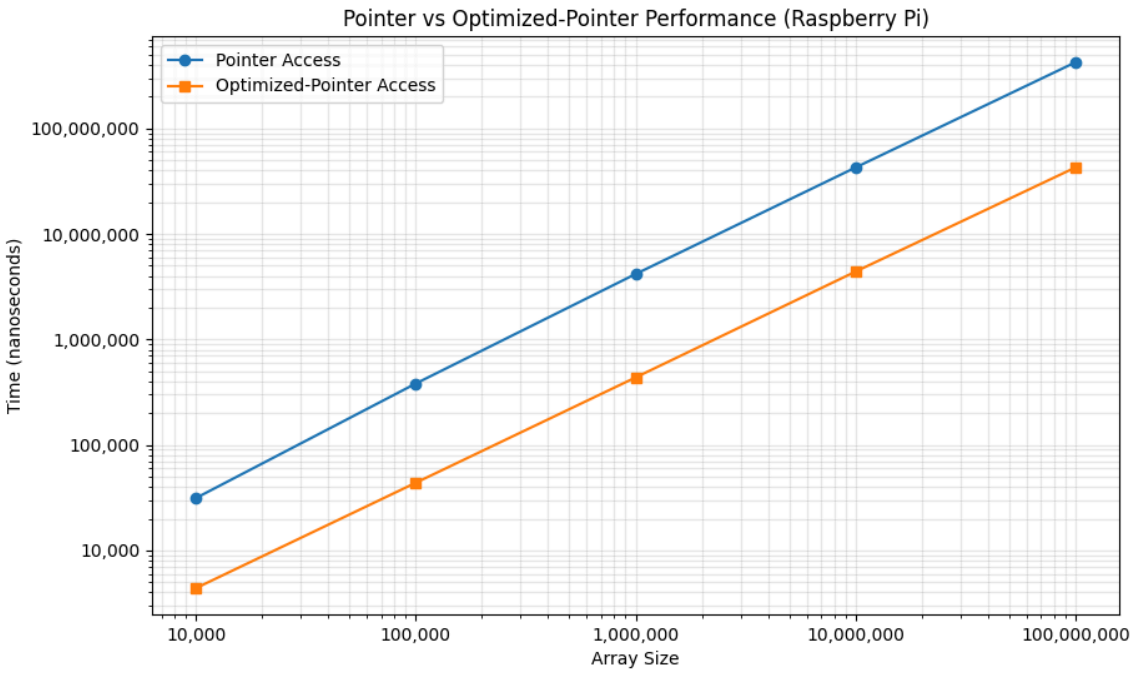

Figure 6. *Raspberry Pi timing comparison of pointer-based vs manually optimized pointer-based array clearing functions of compiler-generated code with no optimization flags.*

Linux Intel Performance Analysis

Linux Intel s File (Snippet): Original ClearUsingIndex()

```
        .text
        .globl _Z15ClearUsingIndexPii
        .type  _Z15ClearUsingIndexPii, @function
_Z15ClearUsingIndexPii:
.LFB2308:
        .cfi_startproc
```

```
        endbr64
        pushq  %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq   %rsp, %rbp
        .cfi_def_cfa_register 6
        movq   %rdi, -24(%rbp)
        movl   %esi, -28(%rbp)
        movl   $0, -4(%rbp)
        jmp    .L5
.L6:
        movl   -4(%rbp), %eax
        cltq
        leaq   0(,%rax,4), %rdx
        movq   -24(%rbp), %rax
        addq   %rdx, %rax
        movl   $0, (%rax)
        addl   $1, -4(%rbp)
.L5:
        movl   -4(%rbp), %eax
        cmpl   -28(%rbp), %eax
        jl     .L6
        nop
        nop
        popq   %rbp
        .cfi_def_cfa 7, 8
        ret
        .cfi_endproc
.LFE2308:
        .size  _Z15ClearUsingIndexPii, .-_Z15ClearUsingIndexPii
```

Code Snippet Explanation

- `pushq %rbp` : Save base pointer for stack frame setup.
- `movq %rsp, %rbp` : Set up a new base pointer for local variables.
- `movq %rdi, -24(%rbp)` : Store first parameter (int* arr) on stack.
- `movl %esi, -28(%rbp)` : Store second parameter (int size) on stack.
- `movl $0, -4(%rbp)` : Initialize local variable i = 0.
- `jmp .L5` : Jump to condition check at end of loop.
- `movl -4(%rbp), %eax` : Load loop index i.
- `cltq` : Sign-extend 32-bit i to 64-bit for address calculation.
- `leaq 0(,%rax,4), %rd, x` : Multiply i by 4 (since each int = 4 bytes).
- `movq -24(%rbp), %rax` : Load base pointer of array.
- `addq %rdx, %rax` : Compute address of array[i].
- `movl $0, (%rax)` : Store 0 into array[i].
- `addl $1, -4(%rbp)` : Increment loop counter i++.
- `movl -4(%rbp), %eax` : Load i.
- `cmpl -28(%rbp), %eax` : Compare i with size.
- `jl .L6` : Jump back to loop if i < size.

Issues:

- Recomputes the address each loop.
- cltq and leaq each iteration = index × 4 overhead.
- Heavy stack usage and poor register reuse.

Linux Intel s File (Snippet): Optimized ClearUsingIndex()

```
_Z15ClearUsingIndexPii:
    endbr64
    testl   %esi, %esi
    jle     .Ldone

    movq    %rdi, %rax
    movl    %esi, %ecx

.Lloop:
    movl    $0, (%rax)
    addq    $4, %rax
    subl    $1, %ecx
    jne     .Lloop

.Ldone:
    ret
       .size  _Z15ClearUsingIndexPii, .-_Z15ClearUsingIndexPii
```

Explanation of Optimized Code Block

- `testl %esi, %esi` : if size <= 0, skip
- `movq %rdi, %rax` : rax = array pointer
- `movl %esi, %ecx` : ecx = size
- `movl $0, (%rax)` : *array = 0
- `addq $4, %rax` : move to next element
- `subl $1, %ecx` : size - -
- `jne .Lloop` : if not zero, continue

Results:
- Removed stack frame setup which voids memory overhead.
- Used pointer increment replacing index multiplication.
- Stored size in register keeps value in fast-access CPU register.
- Decrement loop simplifying comparison and branch logic.

Linux Intel s File (Snippet): Original ClearUsingPointers()

```
      .text
      .globl _Z18ClearUsingPointersPii
      .type  _Z18ClearUsingPointersPii, @function
```

```
_Z18ClearUsingPointersPii:
.LFB2308:
	.cfi_startproc
	endbr64
	pushq  %rbp
	.cfi_def_cfa_offset 16
	.cfi_offset 6, -16
	movq   %rsp, %rbp
	.cfi_def_cfa_register 6
	movq   %rdi, -24(%rbp)
	movl   %esi, -28(%rbp)
	movq   -24(%rbp), %rax
	movq   %rax, -8(%rbp)
	jmp    .L5
.L6:
	movq   -8(%rbp), %rax
	movl   $0, (%rax)
	addq   $4, -8(%rbp)
.L5:
	movl   -28(%rbp), %eax
	cltq
	leaq   0(,%rax,4), %rdx
	movq   -24(%rbp), %rax
	addq   %rdx, %rax
	cmpq   %rax, -8(%rbp)
	jb     .L6
	nop
	nop
	popq   %rbp
	.cfi_def_cfa 7, 8
	ret
	.cfi_endproc
.LFE2308:
	.size  _Z18ClearUsingPointersPii, .-_Z18ClearUsingPointersPii
```

Code Snippet Explanation

- `pushq %rbp / movq %rsp, %rbp` : Standard frame prologue which saves previous base pointer and sets up a new frame (so this function uses a stack frame).
- `movq %rdi, -24(%rbp)` : Save the first argument (pointer Array) to the local stack slot at -24(%rbp). On x86_64 SysV, the first arg pointer is in rdi.
- `movl %esi, -28(%rbp)` : Save the second argument (int size) to -28(%rbp). The second arg is in esi.
- `movq -24(%rbp), %rax / movq %rax, -8(%rbp)` : Load Array from stack and store it at -8(%rbp); this -8(%rbp) slot is used as the current pointer p.
- `jmp .L5` : go check loop condition first.
- `movq -8(%rbp), %rax` : load current p into RAX.
- `movl $0, (%rax)` : store 0 (32-bit) at *p (clear the integer).
- `addq $4, -8(%rbp)` : increment the stored pointer p by 4 bytes (advance to next int).
- `movl -28(%rbp), %eax` : load size into EAX.
- `cltq` : sign-extend EAX to RAX (so RAX = (int64)size).
- `leaq 0(,%rax,4), %rdx` : compute size * 4 into RDX.

- `movq -24(%rbp), %rax` : load base pointer Array into RAX.
- `addq %rdx, %rax` : compute Array + size*4 => &Array[size] (end pointer) in RAX.
- `cmpq %rax, -8(%rbp)` : compare end with current pointer p stored at -8(%rbp).
- `jb .L6` : if p < end (unsigned compare), jump to loop body.
- `popq %rbp / ret` : epilogue + return.

Issues:

- The function creates a full stack frame and spills both arguments and the current pointer into memory (-24, -28, -8) instead of keeping them in registers creating extra memory traffic.
- The loop computes end = Array + size*4 on every iteration (because it's in the .L5 condition block loaded from stack each time) rather than once.
- The loop increments the pointer by storing back to memory (addq $4, -8(%rbp)) each iteration more memory access.
- The store uses movl $0, (%rax) which is fine, but there is no use of rep stosl or other optimized string/memory ops that x86 provides.
- Overall: more instructions / memory accesses per iteration than necessary.
- This is part of the reason that this pointer version underperformed against the index version.

Linux Intel s File (Snippet): Optimized ClearUsingPointers()

```
_Z18ClearUsingPointersPii:
.LFB2308:
	.cfi_startproc
	endbr64
	testl	%esi, %esi
	je	.Ldone_x64
	cld
	xorl	%eax, %eax
	movl	%esi, %ecx
	rep	stosl	%es:(%rdi)
.Ldone_x64:
	ret
	.cfi_endproc
	.size	_Z18ClearUsingPointersPii, .-_Z18ClearUsingPointersPii
```

Explanation of Optimized Code Block

- `testl %esi, %esi` : if size == 0, skip
- `cld` : clear direction flag (forward)
- `xorl %eax, %eax` : zero the value to store
- `movl %esi, %ecx` : set counter
- `rdi` : contains pointer to start

- `rep stosl %es:(%rdi)` : store EAX (0) ECX times to [RDI], advances RDI by 4 each time

Results:

- No stack frame which means fewer pushes/pops and no memory spills.
- A single rep stosl does work in a tight microcoded loop with fewer instructions executed overall.
- Stores are batched, allowing the microarchitecture to optimize writes (store-combining, write buffers).
- For big arrays this will be significantly faster and simpler than per-element load/store pointer arithmetic and frequent memory accesses to -8(%rbp).

Raspberry Pi - Index vs Pointer Timed Results (Optimized)

Table 8. Linux Intel (Optimized) - Index vs. Pointer Performance (Average)

Array Size	Index (ns)	Pointer (ns)
10000	3835.9	636.6
100000	35011.7	11250.1
1000000	419999.9	234156.9
10000000	3938150.2	2672380.6
100000000	37888938.9	28709668.8

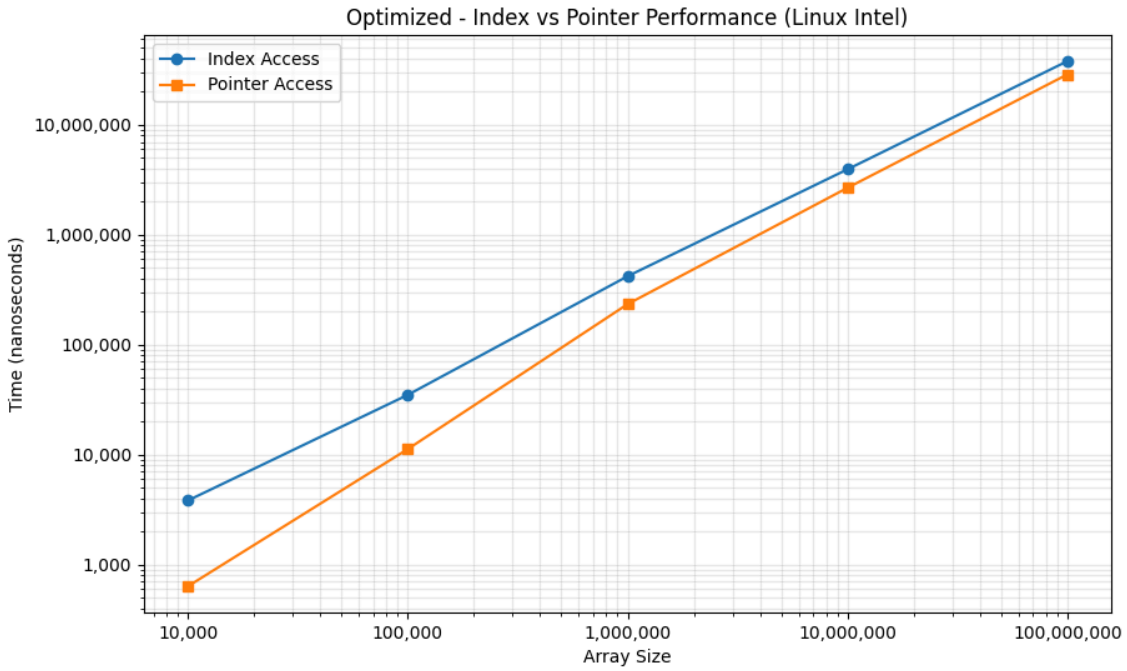

Figure 7. *Linux Intel timing comparison of index-based vs pointer-based array clearing functions using manual optimization of compiler-generated code with no optimization flags.*

Table 9. Linux Intel - Index vs. Index (Optimized) Performance (Average)

Array Size	Original (ns)	Optimized (ns)
10000	15792	3835.9
100000	75273.3	35011.7
1000000	1075911.8	419999.9
10000000	15422548	3938150.2
100000000	153523296.1	37888938.9

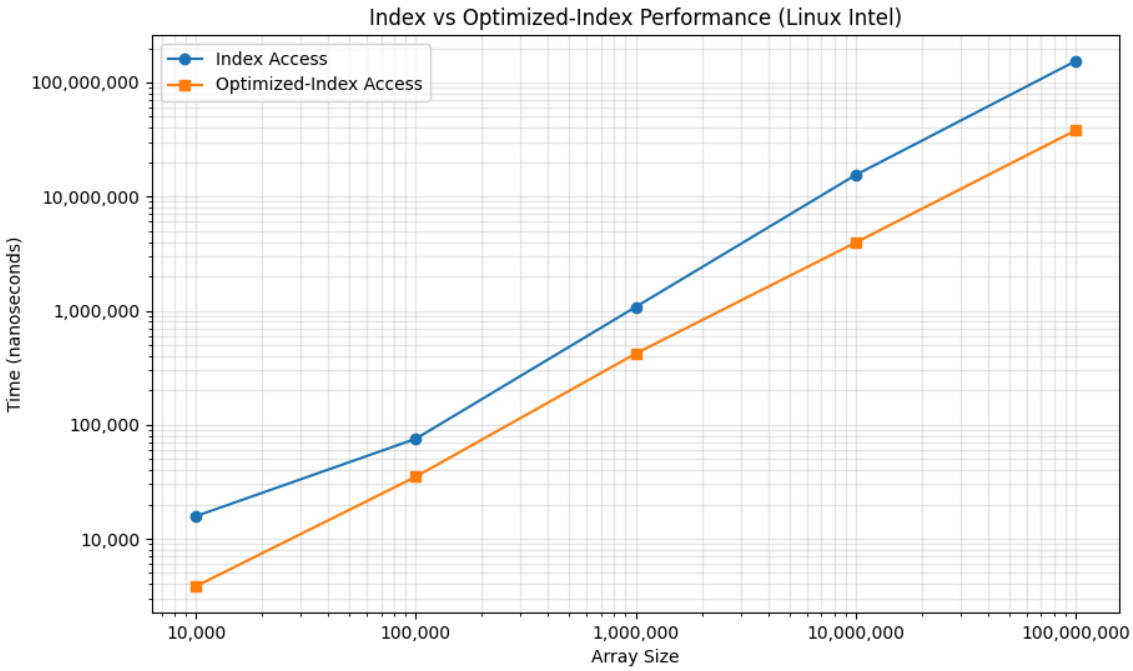

Figure 8. *Linux Intel timing comparison of index-based vs manually optimized index-based array clearing functions of compiler-generated code with no optimization flags.*

Table 10. Linux Intel - Pointer vs. Pointer (Optimized) Performance (Average)

Array Size	Original (ns)	Optimized (ns)
10000	9126.4	636.6
100000	98875.6	11250.1
1000000	1477055.7	234156.9
10000000	15488939	2672380.6
100000000	122375688.2	28709668.8

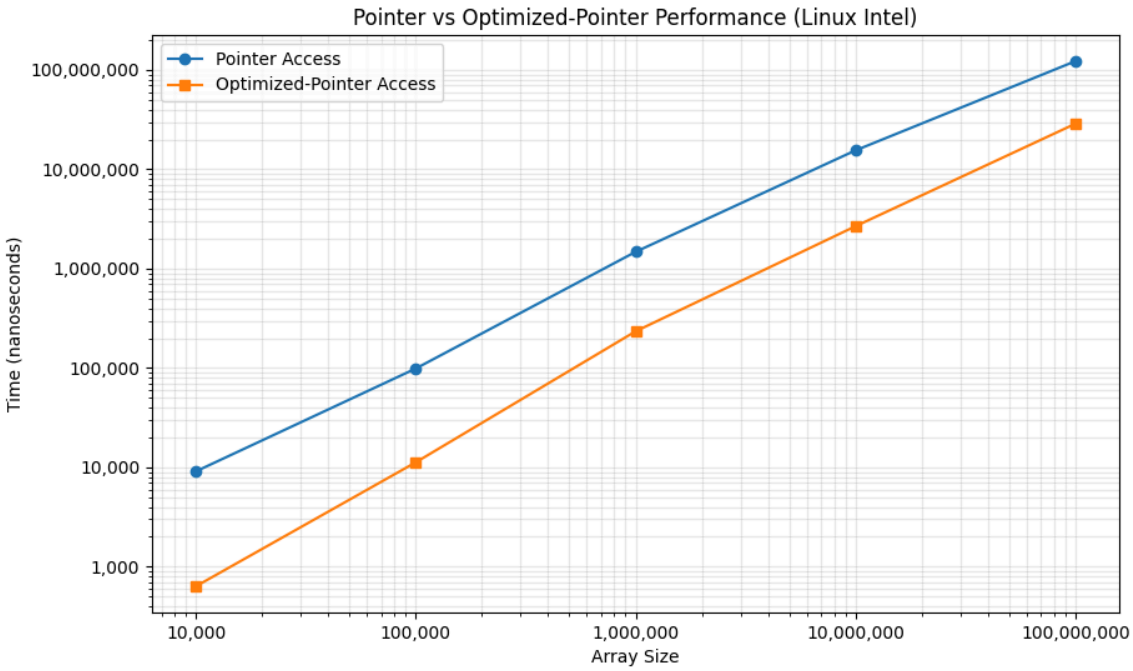

Figure 9. *Linux Intel timing comparison of pointer-based vs manually optimized pointer-based array clearing functions of compiler-generated code with no optimization flags.*

Optimization Summary

Table 11. Index and Pointer Concept Summary

Platform	Function	Key Optimization	Benefit
Raspberry Pi	Index	Pointer-post increment	Fewer loads/stores
Raspberry Pi	Pointer	Already optimal, reduced number of instructions	Minimal Instructions
Linux Intel	Index	Use registers + pointer	Reduced micro-ops
Linux Intel	Pointer	rep stosl to clear + increment	Uses CPU fill hardware

Why Pointer Still Beats Index:

- Address arithmetic happens implicitly → fewer instructions.
- CPU caches work better with sequential pointer increments.
- No loop-carried multiply, lowering latency.
- Compiler auto-vectorization favors pointer form (memset, rep stosl).

Even after hand-optimizing the index loop, pointer-based loops achieve:

- Higher instruction-per-cycle (IPC) ratio.
- Better out-of-order execution.
- Lower energy per iteration.

Appendix A – Raw Timing Data (Raspberry Pi)

- Table A.1: ClearUsingIndex (Original)
- Table A.2: ClearUsingPointers (Original)
- Table A.3: ClearUsingIndex (Optimized)
- Table A.4: ClearUsingPointers (Optimized)

Appendix B – Raw Timing Data (Linux Intel)

- Table B.1: ClearUsingIndex (Original)
- Table B.2: ClearUsingPointers (Original)
- Table B.3: ClearUsingIndex (Optimized)
- Table B.4: ClearUsingPointers (Optimized

Table A.1: ClearUsingIndex Timing Results (Original)

Array Size	10000	100000	1000000	10000000	100000000
Run 1	36759	545509	7851964	51601540	516379743
Run 2	26852	323560	4589761	50491665	504971367
Run 3	24556	317505	5071027	51148536	504935885
Run 4	24371	492342	5103378	47399229	504198616
Run 5	24389	316950	5259252	50748944	506965147
Run 6	24371	338227	5287066	50762551	507482142
Run 7	24797	388321	5053802	50494351	504907772
Run 8	24464	389358	5146229	51682180	516254766
Run 9	24371	592509	4744481	49208489	505052520
Run 10	24871	387599	4839649	51148181	516007233
Average	**25980.1**	**409188**	**5294660.9**	**50468566.6**	**508715519.1**

Table A.2: ClearUsingPointers Timing Results (Original)

Array Size	10000	100000	1000000	10000000	100000000
Run 1	40723	651582	5248677	45966896	416083877
Run 2	22778	405950	4194683	42447126	425832950
Run 3	37593	414024	4075255	41475815	415880315
Run 4	37630	375152	4184182	42559214	425813055
Run 5	37593	404246	4160699	41676956	415961821
Run 6	23371	243429	3937066	43571199	417496006
Run 7	25519	255503	3762694	42757223	432395351
Run 8	23018	255596	3898565	41578002	416091663
Run 9	41241	563729	4287311	42720124	447929736
Run 10	24204	249614	3941491	41188195	416022087
Average	**31367**	**381882.5**	**4169062.3**	**42594075**	**422950686.1**

Table A.3: ClearUsingIndex Timing Results (Optimized)

Array Size	10000	100000	1000000	10000000	100000000
Run 1	6389	62574	625243	4179499	42111797
Run 2	4259	41834	417039	4451279	44169491
Run 3	4241	41722	416854	4171629	42030167
Run 4	4240	41760	416891	4372001	42018815
Run 5	4277	41722	416965	4453298	45507608
Run 6	4259	41704	417094	4476908	42164593
Run 7	4241	41926	419002	4259315	42184648
Run 8	4259	41796	427094	4530113	44525159
Run 9	4241	41741	424428	4316723	42125556
Run 10	4259	41741	416909	4276907	42210353
Average	**4466.5**	**43852**	**439751.9**	**4348767.2**	**42904818.7**

Table A.4: ClearUsingPointers Timing Results (Optimized)

Array Size	10000	100000	1000000	10000000	100000000
Run 1	5666	55648	555891	5266412	42079849
Run 2	4259	41833	443336	4223777	42264331
Run 3	4259	41741	417039	4176111	42184905
Run 4	4259	41741	416928	4291408	44405192
Run 5	4241	41722	421816	4374834	41851329
Run 6	4240	41722	424706	4176999	42134941
Run 7	4259	41722	417113	4306630	42115441
Run 8	4241	41722	416854	4408815	41939069
Run 9	4259	41778	421057	4220703	42147886
Run 10	4241	48315	427150	4466149	44856619
Average	**4392.4**	**43794.4**	**436189**	**4391183.8**	**42597956.2**

Table B.1: ClearUsingIndex Timing Results (Original)

Array Size	10000	100000	1000000	10000000	100000000
Run 1	13755	73693	836414	8726928	204004903
Run 2	19422	74336	788027	10618361	127783340
Run 3	9455	74971	856668	14336800	201992745
Run 4	19368	74098	967960	18896155	122879523
Run 5	19413	74937	861172	17309929	89604894
Run 6	15289	74224	793081	15060710	149360491
Run 7	19601	73803	849697	9585560	212062551
Run 8	19762	74751	813441	20424848	190979046
Run 9	8092	84133	847106	14387543	137843993
Run 10	13763	73787	3145552	24878646	98721475

Average	**15792**	**75273.3**	**1075911.8**	**15422548**	**153523296.1**

Table B.2: ClearUsingPointers Timing Results (Original)

Array Size	**10000**	**100000**	**1000000**	**10000000**	**100000000**
Run 1	12158	213029	1035771	13167129	115857675
Run 2	8980	73216	828217	31645785	95296576
Run 3	9237	100443	801705	9499803	93835615
Run 4	8420	74435	949978	13665192	130855499
Run 5	9087	73359	909866	26099131	87416925
Run 6	7923	73154	923341	9675423	135192561
Run 7	8619	75020	5814163	15443752	138636410
Run 8	8353	73906	927634	9671272	132880143
Run 9	9575	97293	1438755	16470284	86967037
Run 10	8912	134901	1141127	9551619	206818441
Average	**9126.4**	**98875.6**	**1477055.7**	**15488939**	**122375688.2**

Table B.3: ClearUsingIndex Timing Results (Optimized)

Array Size	**10000**	**100000**	**1000000**	**10000000**	**100000000**
Run 1	3472	32445	327738	3788890	38006478
Run 2	4237	43713	372108	3508301	35951226
Run 3	3938	31396	645579	3752402	36787217
Run 4	3399	32265	426041	3630676	36965022
Run 5	4467	34656	377696	3588841	39753627
Run 6	3434	32399	350240	3809096	38546417
Run 7	3490	32441	485418	3805174	38451703
Run 8	4501	39773	401261	5541581	37315308
Run 9	3993	38626	439242	4203167	39275398

Run 10	3428	32403	374676	3753374	37836993
Average	**3835.9**	**35011.7**	**419999.9**	**3938150.2**	**37888938.9**

Table B.4: ClearUsingPointers Timing Results (Optimized)

Array Size	**10000**	**100000**	**1000000**	**10000000**	**100000000**
Run 1	659	12518	259025	2545898	28825874
Run 2	589	25810	211247	2579633	29020911
Run 3	786	14811	237015	2625744	28551059
Run 4	595	7558	210091	2745540	29863100
Run 5	597	8400	248599	2697018	28064120
Run 6	633	8642	279920	2716295	28672627
Run 7	739	8334	249192	2816109	28647248
Run 8	593	9811	240867	2575564	28599372
Run 9	567	8337	193693	2745339	28607712
Run 10	608	8280	211920	2676666	28244665
Average	**636.6**	**11250.1**	**234156.9**	**2672380.6**	**28709668.8**